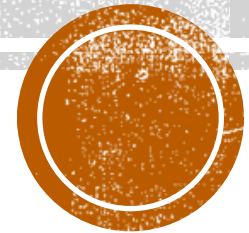




Module: Développer en Front-end

Gestion d'évènements-Communication entre Composants-Cycle de vie des Composants



Filière: Développement digital – option web full stack

Réalisé par: ELMAUDNI

Gestion d'évènements

- **Gestion des événements** : La gestion des événements avec des éléments React est très similaire à la gestion des événements sur les éléments DOM. Il existe quelques différences de syntaxe :
- Les événements React sont nommés en camelCase, plutôt qu'en minuscules. Ex
- **onclick en HTML ➔ onClick en React**
- Avec JSX, vous transmettez une fonction en tant que callback fonction ou arrow fonction, plutôt qu'une chaîne.
- **onclick="ajouter()" en HTML ➔ en React :**
- **onClick={ajouter}** : callback function
- **onClick={()=>ajouter()}: Arrow function**

Communication inter-composant (envoi, réception de données):

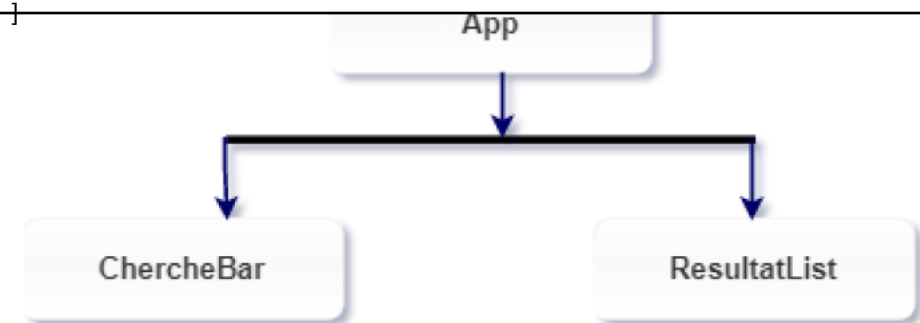


On suppose que nous disposons de trois composants :

- App
- ChercherBar
- ResultatList

Les composants ChercherBar et ResultatList sont imbriqués dans le composant App

```
const list=[  
  {nom:"banane",type:"fruit"},  
  {nom:"orange",type:"fruit"},  
  {nom:"pomme",type:"fruit"},  
  {nom:"raisins",type:"fruit"},  
  {nom:"carotte",type:"legume"},  
  {nom:"pomme de terre",type:"legume"},  
  {nom:"navet",type:"legume"},  
  {nom:"poivron",type:"legume"}  
]
```



- Composant App:

```
export default function App() {
  const [type,setType]=useState("")
  const [resultas,setResultas]=useState([])
  function onChercheSubmitApp(type){
    setType(type)
    setResultas(list.filter((item)=>item.type.toUpperCase()===type))
  }
  return (
    <div className='App'>
      <h1>Composant App</h1>
      <ChercheBar onChercheSubmit={onChercheSubmitApp}/>
      <div>
        <p>le type:<span style={{color:"rgb(36,
        44,33)",fontWeight:"bold"}}>{type}</span></p>
      </div>
      <ResultatList resultats={resultas}/>
    </div>
  );
}
```

```

import React, { useState } from "react";
export default function ChercheBar(props) {
  const [term,setTerm]=useState("")
  function onChercheSubmitBar(event){
    event.preventDefault()
    props.onChercheSubmit(term)
  }
  return (
    <div className="Child">
      <form
        onSubmit={(event)=>onChercheSubmitBar(event)}>
        <h2>composant ChercheBar</h2>
        <div>
          <label>Entrer le mot clé de recherche:</label>
          <input type="text" value={term}
            onChange={(event)=>setTerm(event.target.value.toUpp
              erCase())} />
        </div>
        <button type="submit">chercher</button>
      </form>
    </div>
  );
}

```

```

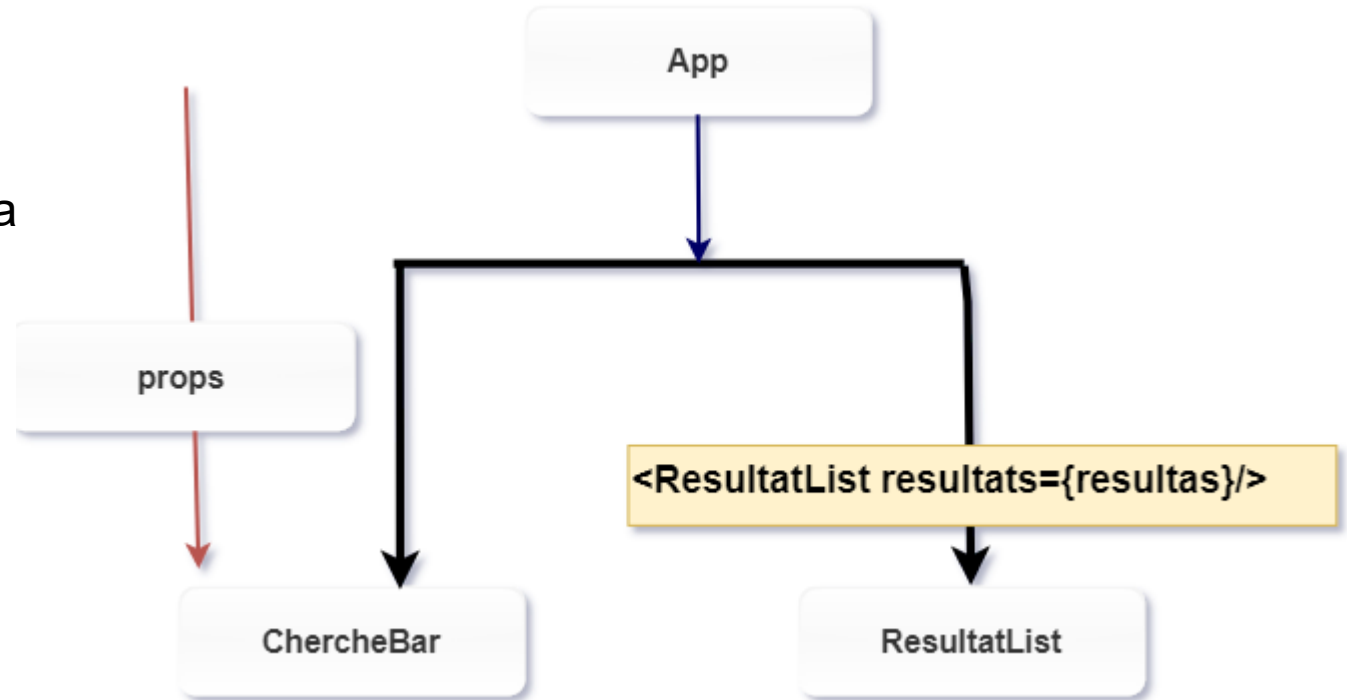
import React from "react";
export default function ResultatList(props) {
  return (
    <div className="Child">
      <h1>composant ResultatList</h1>
      {props.resultats.length == 0 ? (
        <p>pas de resultats</p>
      ) : (
        <div className="list">
          <ul>
            {props.resultats.map((item) => {
              return <li key={item.nom}>{item.nom}</li>;
            })}
          </ul>
        </div>
      )}
    </div>
  );
}

```

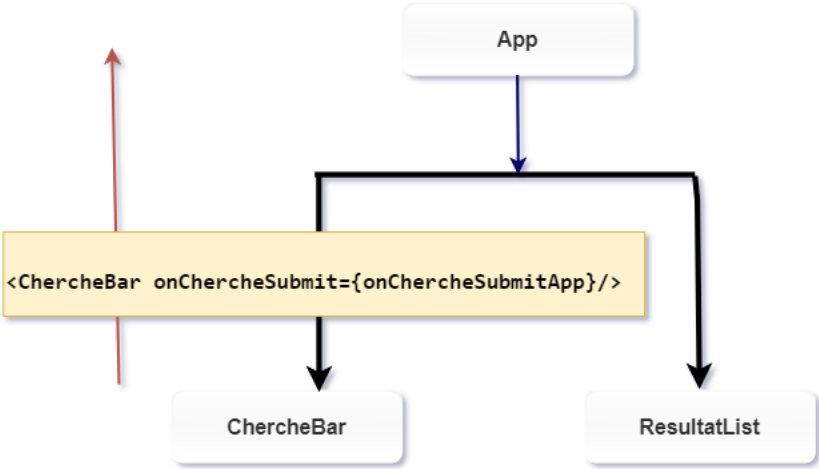
Communication de parent vers enfant

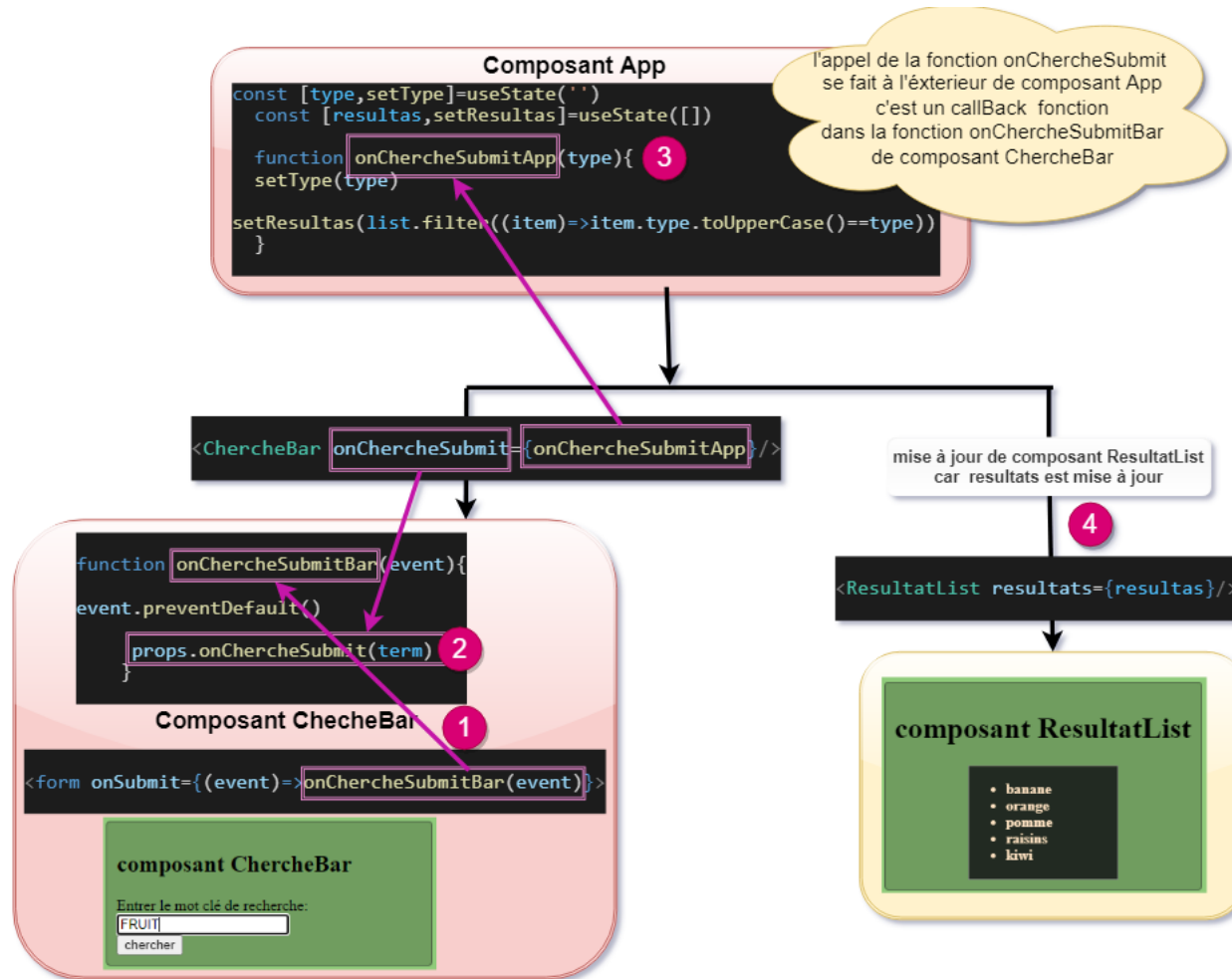
`<ResultatList resultats={resultas}/>`

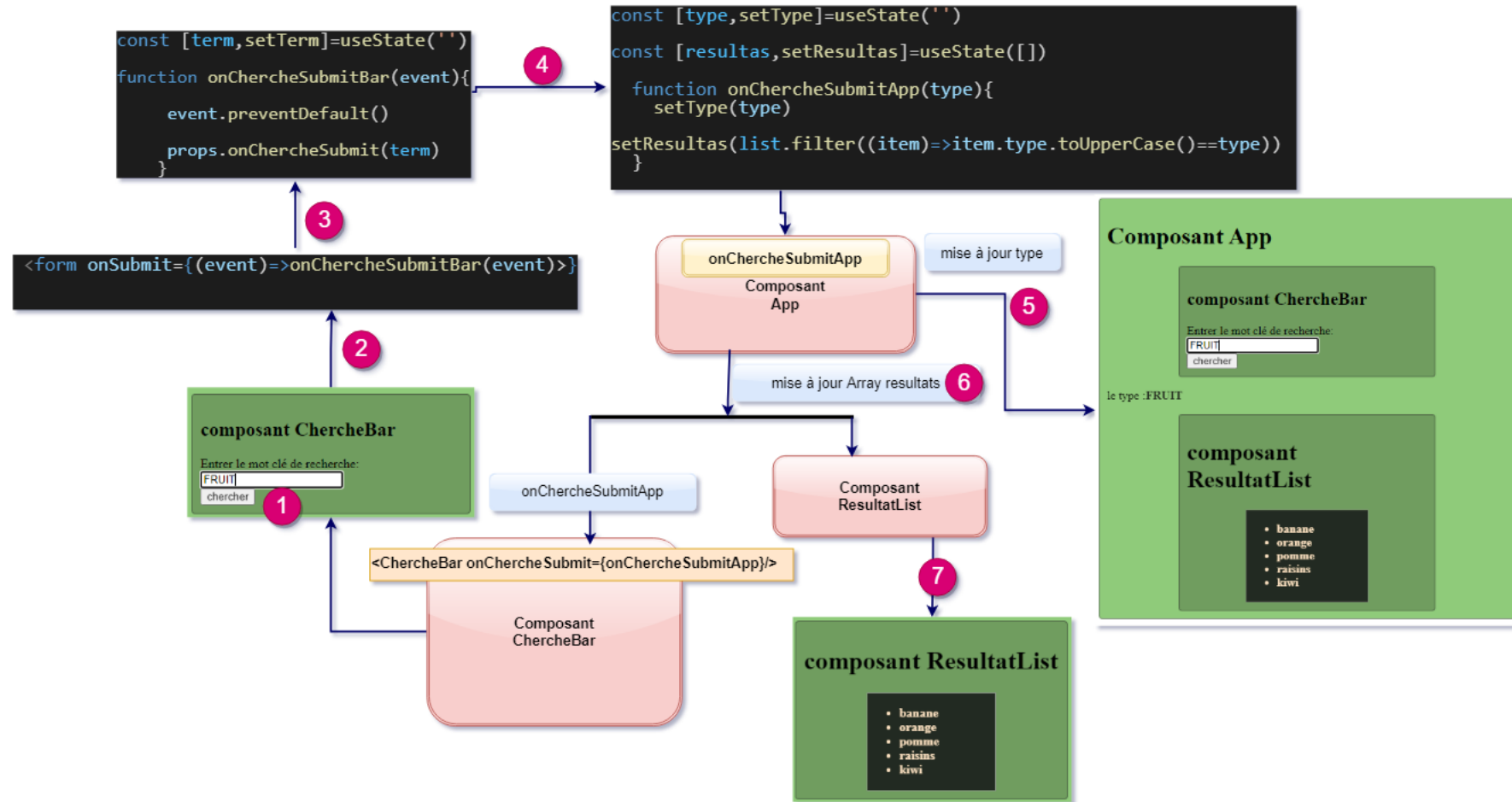
Le composant ResultatList reçoit l'informa
le props resultats



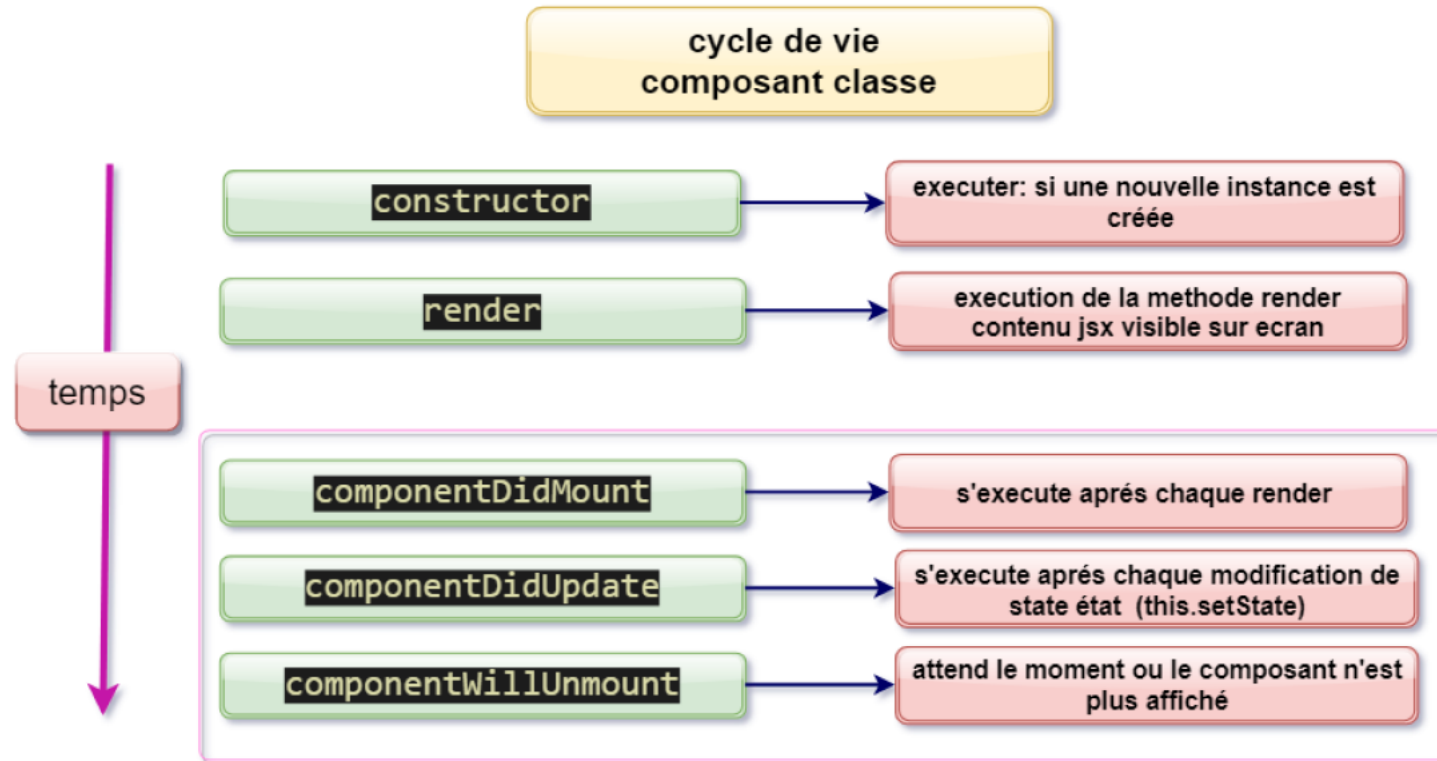
Communication d'enfant vers parent







Cycle de vie des composants



Cycle de vie Composants créer via fonction

